

Smart Farm IoT Monitoring and Alert System

Project Report

Internet of Things — A.Y. 2024/2025

University of Cagliari — DIEE

Supervised by: Prof. Michele Nitti

Students:

Ghita El Belghiti

Essil Amaimi

Table of Contents

Table of Contents.....	2
1. Introduction	3
2. Planning Phase	3
2.1 System Request and Motivation	3
2.2 Feasibility Analysis	3
3. Analysis Phase	4
3.1 Reference Scenario	4
3.2 State of the Art — Competitor Analysis.....	4
3.3 Functional Requirements.....	5
3.4 Non-Functional Requirements.....	7
3.5 Technology Analysis	7
3.5.1 Sensors	7
3.5.2 Microcontroller / Development Board	8
3.5.3 Communication Pattern	8
3.5.4 Communication Protocol	9
3.5.5 User Interface.....	9
4. Design Phase	10
4.1 System Architecture.....	10
4.2 Database Design.....	11
4.3 Data Exchange Format	11
4.4 Hardware Wiring Design	11
4.5 Software Flowcharts	12
4.5.1 ESP8266 Data Collection and Transmission (FR1, FR2).....	12
4.5.2 Subscriber and Storage (FR3).....	12
4.5.3 Dashboard Alert Algorithm (FR6).....	12
4.5.4 Actuator Control Logic (FR7)	12
4.5.5 Telegram Alert Dispatch (FR8)	12
4.6 Digital Twin Concept	13
4.7 Application Layer: Dashboard and Telegram Bot	13
4.8 Flow of Operation	13
5. Demo Description	14
5.1 Configuration	14
5.2 Demo Flow	14
6. Conclusion.....	14

1. Introduction

This report documents the design and development of a Smart Farm IoT Monitoring System. The project was developed as part of the Internet of Things course and follows the System Development Life Cycle (SDLC) methodology.

The system collects environmental data from a set of sensors connected to an ESP8266 microcontroller, transmits it to a local server, stores it for historical analysis, and displays it through a web dashboard. In addition, the system is able to react to critical conditions by activating a physical actuator and by pushing alert notifications to the farmer through a Telegram bot, closing the loop between sensing and action.

The project was developed by two students. One handled the hardware side — sensor wiring, ESP8266 setup, actuator control, and Arduino code — while the other handled the software side, including the message subscriber, database, backend API, dashboard, and the Telegram bot integration.

2. Planning Phase

2.1 System Request and Motivation

Agriculture is one of the sectors where IoT can make the biggest difference. Small farms and greenhouses often rely on manual checks to monitor environmental conditions like temperature, humidity, soil moisture, and light. This is not only time-consuming but also unreliable, since conditions can change quickly and go unnoticed.

The idea behind this project is to build a low-cost monitoring system that automates this process and is able to react on its own to critical conditions, rather than only display them. The system should collect sensor data continuously, display it in a way that allows a farmer or researcher to check conditions from a browser without needing to be physically present, activate an actuator when a condition requires an immediate physical response, and notify the farmer proactively rather than requiring them to keep the dashboard open.

2.2 Feasibility Analysis

Technical feasibility: The ESP8266 is a widely available and well-documented microcontroller with built-in WiFi, which makes it suitable for this kind of project. The sensors selected are inexpensive and compatible with the board, and the software stack is open source and runs on any laptop.

Economic feasibility: The hardware cost is very low — the microcontroller, sensors, and actuator module can be purchased for a small total cost. The software stack is entirely free and open source, and the Telegram Bot API used for notifications is free of charge.

Organizational feasibility: The system is designed to be simple to deploy. The dashboard is accessible from any browser without installing anything on the client side, and alerts reach the user directly on a messaging app they already use.

3. Analysis Phase

3.1 Reference Scenario

The scenario we are targeting is a small agricultural environment, such as a greenhouse or an experimental farm plot. The people involved are students or researchers who need to monitor plant growth conditions without always being physically present.

The environment is indoor or semi-outdoor. The sensors need to track four key parameters: temperature, air humidity, soil moisture, and light intensity — the main environmental variables that affect plant health and irrigation needs. When soil moisture drops below a safe level, the system should be able to act directly on the environment) rather than only reporting the problem.

The system is expected to be used continuously, logging data every few seconds, and the dashboard should be accessible from any device connected to the local network. Alerts should be generated automatically, both visually on the dashboard and remotely through a push notification, when values go outside acceptable ranges.

Assumptions made for this project:

- The ESP8266 and the server machine are connected to the same WiFi network.
- The MQTT broker (Mosquitto) runs locally on the server machine.
- No cloud connectivity is required for the core monitoring loop — the system works entirely on a local network; only the Telegram notification channel reaches outside the local network, using the public Telegram Bot API.
- A single ESP8266 node is used, monitoring one location and controlling one actuator.
- Data is stored locally — no external database server is needed.

3.2 State of the Art — Competitor Analysis

Before defining our system, we looked at existing solutions in the smart farm monitoring space to understand what already exists, how they are priced, and what we could do differently.

System	Key Features	Advantages	Disadvantages	Cost
Blynk IoT Platform	Cloud-based dashboard, mobile app, supports multiple boards	Easy to set up, good UI, mobile access	Requires internet, data stored on third-party servers	Free tier limited to 2 devices; paid plans start around 5-25 USD/month for more devices and higher data rates
Node-RED + InfluxDB + Grafana	Visual flow editor, time-series database, powerful charts	Very flexible, open source, good for complex setups	Complex to configure, steep learning curve, heavy stack for small projects	Free and open source, but requires more powerful hardware to self-host (higher indirect cost)
ThingSpeak	Cloud IoT platform with MATLAB analytics, free tier available	Simple API, free basic use, built-in chart tools	Limited to 15 second update interval on free plan, data on external server, less customizable	Free tier available; paid "Home" and "Standard" licenses required for faster updates or commercial use

From this analysis, we can identify two main categories of existing systems: cloud-based platforms (Blynk, ThingSpeak), which are easy to use but depend on external servers, internet connectivity, and often a paid tier once the free quota is exceeded, and self-hosted stacks (Node-RED), which are very powerful and free of licensing costs but overly complex, and computationally heavier, for a simple monitoring use case.

Our system sits in between: it is fully local like a self-hosted solution, but much simpler to deploy, and it has no recurring licensing cost since every component (Mosquitto, Flask, our database, Chart.js, and the Telegram Bot API) is free. All monitoring data stays on the local machine, and the dashboard is custom-built to show exactly what is needed without unnecessary complexity.

Comparing our design choices against this state of the art:

- What is standard across all reviewed systems, and that we also adopted: continuous sensor polling, a web-based dashboard for visualization, and threshold-based alerting.
- What we added that most reviewed systems do not offer out of the box: a physical actuator directly controlled by the same threshold logic that drives the alerts, closing the sense-decide-act loop instead of stopping at notification; and a push-notification channel (Telegram) that reaches the user proactively, rather than requiring them to keep a dashboard or mobile app open.
- What we deliberately left out compared to platforms like Blynk or Node-RED: multi-device fleet management, cloud storage, and a visual flow-programming environment — all of which add complexity and cost that is not justified for a single-node monitoring scenario like ours.

3.3 Functional Requirements

The functional requirements describe what the system must be able to do. They are described independently of the specific technology used to implement them. The technology choices made to satisfy each requirement are discussed and justified in section 3.5, Technology Analysis.

FR1 — Sensor Data Acquisition	
Description	The system is able to collect temperature, humidity, soil moisture, and light intensity readings from the sensors deployed in the monitored environment.
Input	Physical measurements from the temperature/humidity sensor, the soil moisture sensor, and the light sensor.
Output	Raw sensor values available for processing and transmission.

FR2 — Data Transmission	
Description	The system is able to transmit sensor readings from the monitoring node to the server over the local wireless network.
Input	Raw sensor values from FR1.
Output	Structured messages, one per sensor reading, delivered to the server.

FR3 — Data Storage

Description	The system is able to store incoming sensor data persistently on the server so that historical data is available for analysis.
Input	Messages received by the server from FR2.
Output	Records persisted with sensor type, value, and timestamp.

FR4 — Real-Time Dashboard Visualization

Description	The user is able to view the current sensor readings and live-updating charts on a web dashboard without refreshing the page manually.
Input	Request for the latest sensor values.
Output	Live KPI cards and charts, periodically updated, showing current temperature, humidity, moisture, and light values.

FR5 — Historical Data Visualization

Description	The user is able to view historical sensor data filtered by date range on a dedicated history page, with statistical summaries.
Input	Date range and sensor filter selected by the user.
Output	Historical charts and average/min/max statistics for the selected period.

FR6 — Threshold-Based Dashboard Alerts

Description	The system is able to generate visual alerts on the dashboard when sensor values exceed predefined safety thresholds.
Input	Current sensor values from FR4, compared against configured thresholds.
Output	Colored alert banners and a Farm Status indicator (Healthy / Monitor Closely / Attention Required).

FR7 — Automatic Actuator Control

Description	The system is able to activate a physical actuator automatically when a monitored condition (e.g. soil moisture) crosses a critical threshold, and allows the user to trigger or override the actuator manually.
Input	Current sensor values compared against critical thresholds; manual command from the dashboard or Telegram bot.
Output	Actuator state change (ON/OFF) applied to the physical device, and a log entry of the action taken.

FR8 — Remote Push Alerting

Description	The system is able to notify the user proactively, outside of the dashboard, when a sensor value crosses an alert threshold.
--------------------	--

Input	Alert condition detected by the same logic used in FR6/FR7.
Output	A push message delivered to the user's Telegram chat, describing the parameter, its value, and the severity of the event.

3.4 Non-Functional Requirements

ID	Category	Description
NFR1	Update Latency	The dashboard must refresh sensor data within a few seconds of a new reading becoming available.
NFR2	Reliability	The system must continue to function and display data even if the monitoring node is temporarily disconnected, falling back gracefully rather than breaking.
NFR3	Portability	The server-side stack must run on any standard machine without requiring cloud services or a dedicated database server.
NFR4	Usability	The dashboard must be accessible and readable on both desktop and mobile screens without horizontal scrolling, and alerts must be understandable without technical knowledge.
NFR5	Scalability	The data model must allow the addition of new sensor types or a second monitoring node without requiring structural changes to the existing codebase.
NFR6	Cost / Affordability	The total hardware cost per monitoring node, and the recurring cost of any external service used (e.g. notifications), must remain minimal, favoring free and open-source components.
NFR7	Data Persistence	All sensor readings must be stored permanently on the server and must not be lost when the server is restarted.

Of these, Cost/Affordability (NFR6) and Reliability (NFR2) were the requirements that most directly shaped our technology choices: every component was selected first for being free or near-free and for degrading gracefully rather than failing outright, even where a more feature-rich or more scalable alternative existed. Scalability (NFR5) and Portability (NFR3) were treated as secondary — satisfied by keeping the data model flexible and the stack dependency-light, but not actively engineered for multi-node or multi-site deployment, since a single monitored plot was the actual scope of this project.

3.5 Technology Analysis

3.5.1 Sensors

For FR1, each environmental parameter required choosing among alternative sensor types. The comparison below scores the main candidates against the relevant NFRs.

Parameter	Sensor Option	Cost (NFR6)	Reliability (NFR2)	Ease of Integration
Temperature / Humidity	DHT11	Very low	Medium — limited precision, ~1 reading/sec max	High — single digital pin, widely documented
Temperature / Humidity	DHT22	Low	High — better precision and range than DHT11	High — same interface as DHT11

Parameter	Sensor Option	Cost (NFR6)	Reliability (NFR2)	Ease of Integration
Temperature / Humidity	BME280 (I2C)	Medium	High — also provides pressure	Medium — requires I2C wiring
Soil Moisture	Resistive soil moisture sensor	Very low	Low — corrodes over time with constant power	High — simple analog read
Soil Moisture	Capacitive soil moisture sensor	Low	High — no corrosion, more stable readings	High — simple analog read
Light	LDR (photoresistor) + voltage divider	Very low	Medium — non-linear response, needs calibration	High — simple analog read
Light	BH1750 (digital lux sensor, I2C)	Low-Medium	High — calibrated lux output	Medium — requires I2C wiring

We selected the DHT11, a resistive soil moisture sensor, and an LDR, prioritizing NFR6 (cost) and ease of integration, since the ESP8266 only exposes a single analog input and adding I2C sensors for every parameter would have increased both cost and wiring complexity beyond what this project needed. The trade-off, acknowledged explicitly, is lower precision and, for the resistive soil sensor, reduced long-term reliability compared to a capacitive alternative. This is revisited in the Known Limitations discussion.

3.5.2 Microcontroller / Development Board

For FR1 and FR2, we needed a development board with WiFi capability and enough analog/digital pins to connect all sensors and the actuator. We compared three options against the relevant NFRs.

Board	Built-in WiFi (FR2)	Cost (NFR6)	Reliability (NFR2)	Ease of Use
Arduino UNO	No — needs external WiFi shield	Medium (board + shield)	High	High
ESP8266 (NodeMCU)	Yes	Very low	Medium — single analog input, occasional WiFi drops under load	High — well supported in Arduino IDE
Raspberry Pi 3	Yes	High	High	Medium — overkill for simple sensor polling

We chose the ESP8266 (NodeMCU v3). It satisfies FR2 with native WiFi, it best satisfies NFR6 among the three options, and it remained easy to program within the project timeline. The Arduino UNO was rejected because it needs an external WiFi module, which raises both cost and integration effort, and the Raspberry Pi was rejected as unnecessarily powerful and expensive relative to the workload of polling four sensors and one actuator.

3.5.3 Communication Pattern

Before choosing a transport protocol, we evaluated which communication pattern best fits each group of functional requirements, following the same reasoning used for IoT communication patterns (client/server, publish/subscribe, event-driven).

Pattern	Fit for FR2/FR3 (telemetry)	Fit for FR6/FR8 (alerts)	Scalability (NFR5)
Client/Server (request/response)	Possible, but requires the node to know the server address and forces the server to poll or be polled	Poor fit — alerts are event-driven by nature, not something to poll for	Medium — each new node needs direct server knowledge
Publish/Subscribe	Very good — the node publishes readings without knowing who consumes them; any number of subscribers (DB writer, alert engine) can listen independently	Good — an alert can be modeled as a message on a dedicated topic, decoupling detection from delivery	High — new subscribers can be added without touching the publisher
Event-driven (direct callback)	Possible for a single consumer, but tightly couples the node to that consumer	Good for the immediate reaction, but not for keeping a historical log	Low — direct coupling limits reuse

Publish/Subscribe was chosen as the pattern for the whole telemetry and alert pipeline. The ESP8266 publishes readings without needing to know whether the value is going into the database, onto the dashboard, into the alert engine, or triggering the actuator — those are all independent subscribers to the same stream. This directly satisfies NFR5 (Scalability), since a second sensor node or a new type of subscriber can be added without modifying the publisher.

3.5.4 Communication Protocol

Having settled on Publish/Subscribe, we compared the main application protocols available for implementing it.

Protocol	Pattern	Resource Usage	Suitability
HTTP/REST	Client-Server (request/response)	High — full HTTP headers per request	Good for web APIs, not a natural fit for continuous sensor publishing
MQTT	Publish-Subscribe	Very low overhead	Ideal for constrained IoT devices — chosen
CoAP	Client-Server (over UDP)	Low	Good for constrained networks, but less common tooling and no native pub/sub

MQTT was chosen because it is the standard protocol for IoT sensor communication and directly implements the Publish/Subscribe pattern selected above, at very low overhead — an important factor given the ESP8266's limited resources. This also satisfies NFR1 (Update Latency), since it is event-driven rather than polling-based. The broker used is Mosquitto, running locally on the server machine, which also keeps this choice consistent with NFR3 (Portability) and NFR6 (Cost), since no external broker subscription is required.

The REST layer was not discarded, however: the Flask backend still exposes HTTP endpoints, but only at the Service Layer boundary consumed by the dashboard (FR4, FR5) — a request/response pattern that fits a human clicking "load history" far better than it fits a sensor publishing every few seconds.

3.5.5 User Interface

For FR4, FR5, FR6, and FR8, we considered a mobile native app, a Telegram bot, and a web dashboard.

- Mobile app: would require separate development for iOS and Android, out of scope for this project.
- Telegram bot: simple to implement and free of charge, but limited in terms of data visualization and historical charts — not suitable as the only interface.
- Web dashboard: accessible from any device with a browser, fully customizable, supports interactive charts — chosen as the primary interface for FR4, FR5, and FR6.

Rather than choosing a single interface, we adopted a hybrid approach, similar to the pattern used in comparable child- and asset-monitoring IoT systems: the web dashboard remains the primary interface for continuous monitoring and historical analysis (FR4, FR5, FR6), while the Telegram bot is used exclusively as a push channel for FR8, reaching the user even when the dashboard is not open. This plays to the strength of each interface rather than forcing one tool to do both jobs. The web dashboard was built using HTML, CSS, and JavaScript with Chart.js for the charts; the Telegram bot uses the public Telegram Bot API to send messages to a pre-configured chat when the alert engine detects a threshold breach.

4. Design Phase

4.1 System Architecture

The system is organized following a four-layer architecture inspired by the Digital Twin reference model, which separates the physical acquisition of data from its interpretation and its presentation to the user. This replaces a flatter acquisition/communication/application view with an explicit distinction between the raw digital copy of the sensors' state and the logic that turns that state into decisions.

1. Virtualization Layer —

Formed by the ESP8266, the sensors, and the actuator. This layer reads the physical world and builds the Digital Replica: a structured, normalized representation of the current sensor values and actuator state, published as MQTT messages.

2. Service Layer —

Formed by the Mosquitto broker and the Flask REST API. This layer decouples the physical devices from the logic that consumes their data: the broker distributes telemetry to any number of subscribers, and the API exposes that data as callable services (/api/latest, /api/history) without exposing the underlying transport.

3. Digital Twin Layer —

Formed by the Python subscriber and the threshold/alert engine. This is where the raw Digital Replica becomes information: incoming readings are persisted, compared against thresholds, and, when a critical condition is detected, translated into a decision — raise a dashboard alert (FR6), activate the actuator (FR7), and/or send a Telegram notification (FR8).

4. Application Layer —

Formed by the web dashboard and the Telegram bot. This layer does not process data; it presents the state and decisions produced by the Digital Twin Layer to the user, and relays manual commands (e.g. a manual actuator override) back down to the Service Layer.

Figure placeholder: four-layer Digital Twin architecture diagram (Virtualization → Service → Digital Twin → Application), to be inserted here, following the same layout style used for the architecture diagram shown in the course's project design slides.

4.2 Database Design

The database uses a single table to store all sensor and actuator-event readings. The design was chosen to keep the schema simple and flexible — if a new sensor or actuator is added in the future, no schema change is needed.

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique row identifier
topic	TEXT	NOT NULL	Source topic (e.g. farm/temperature, farm/actuator/pump)
value	TEXT	NOT NULL	Reading or actuator state as string
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time the record was stored

One record is inserted for each incoming message, whether it originates from a sensor or from an actuator state change. The backend API reconstructs structured readings by querying each topic separately and merging them by timestamp.

4.3 Data Exchange Format

The ESP8266 publishes a JSON payload to each topic. The sensor format used is:

```
{"value": 24.5}
```

Actuator commands and state reports follow the same convention on a dedicated topic, e.g. farm/actuator/pump, with a payload such as {"state": 1} for ON and {"state": 0} for OFF. Each topic is handled independently by the subscriber, so sensors and the actuator can be added or removed without changing the message format. When the hardware is not yet available, a seed script inserts test data directly into the database using the same schema, so the rest of the stack behaves identically.

4.4 Hardware Wiring Design

The ESP8266 (NodeMCU v3) is connected to the sensors and the actuator as follows. The actuator wiring below (relay-driven water pump, controlled by Essil) reflects the physical build used in the demo — pin numbers should be checked against the actual board layout used before submission.

Component	Pin on ESP8266	Signal Type	MQTT Topic
DHT11 (Temperature)	D4 (GPIO2)	Digital	farm/temperature
DHT11 (Humidity)	D4 (GPIO2)	Digital (same pin)	farm/humidity
Soil Moisture Sensor	A0 (ADC)	Analog	farm/moisture
LDR Light Sensor	A0 (ADC)	Analog (voltage divider)	farm/light
Relay Module (Actuator)	D1 (GPIO5)	Digital output	farm/actuator/pump

Note: the ESP8266 has only one analog input (A0), so the soil moisture and light sensors share this pin and are read sequentially in the Arduino loop with a small delay between them. Power for the sensors is supplied via the USB port during development; the relay module switches the pump's own power line and is not powered from the ESP8266's 3.3V regulator, to avoid overloading it.

4.5 Software Flowcharts

4.5.1 ESP8266 Data Collection and Transmission (FR1, FR2)

- Start: connect to WiFi and MQTT broker.
- Loop every 10 seconds: read DHT11 → validate reading → publish to farm/temperature and farm/humidity.
- Read A0 for soil moisture → publish to farm/moisture.
- Read A0 for light → publish to farm/light.
- If MQTT connection is lost: reconnect before next publish attempt.

4.5.2 Subscriber and Storage (FR3)

- On connect: subscribe to all farm/# topics.
- On message: parse JSON payload → extract value → insert record into the database with topic, value, and current timestamp.
- On error: log the error and continue (non-blocking), so a single malformed message does not stop the pipeline.

4.5.3 Dashboard Alert Algorithm (FR6)

- On each data fetch: compare temperature, humidity, moisture, and light against threshold constants.
- If a value exceeds the danger threshold: mark as "danger" → red alert badge.
- If a value exceeds the warning threshold: mark as "warn" → orange alert badge.
- Farm Status is determined by the worst level among all sensors; alert banners update dynamically without a page reload.

4.5.4 Actuator Control Logic (FR7)

- On each new soil moisture reading: compare against the configured critical-low threshold.
- If moisture is below threshold and the pump is currently OFF: publish an ON command to farm/actuator/pump and log the action.
- If moisture returns above a safe recovery threshold and the pump is currently ON: publish an OFF command and log the action.
- A manual command received from the dashboard or the Telegram bot always overrides the automatic decision until the next automatic evaluation cycle.

4.5.5 Telegram Alert Dispatch (FR8)

- On each alert evaluation (same cycle as FR6): if a sensor value crosses the danger threshold and no notification has been sent for that condition in the last cooldown window, send a message via the Telegram Bot API to the configured chat ID.

- Message content includes the parameter name, the current value, and the threshold that was crossed.
- If the Telegram API call fails (e.g. no internet access), the error is logged and the dashboard alert (FR6) still fires normally, so the core monitoring loop is not affected.

4.6 Digital Twin Concept

In this system, the Digital Twin of the physical farm environment is represented by the database combined with the alert/actuator logic described in section 4.1's Digital Twin Layer. Each physical sensor reading corresponds to a record in the database, and the live dashboard and Telegram bot present a real-time virtual representation of the farm's state — not just its raw values, but the decisions taken because of them.

The dashboard KPI cards show the last known value of each physical sensor, while the line charts show their evolution over time. The Farm Status indicator aggregates all sensor states into a single health indicator, and the actuator's logged ON/OFF history provides a record of how the digital twin has acted on the physical environment over time — closing the loop between sensing, decision, and action.

4.7 Application Layer: Dashboard and Telegram Bot

The dashboard is organized in two pages: Live and History.

Live page layout (top to bottom):

- Navigation bar with SmartFarm brand, page tabs, and live status indicator.
- Alert banner (hidden when all sensors are within the configured safe range).
- Farm Status bar showing overall health.
- Four KPI cards: Temperature, Humidity, Soil Moisture, Light — each with its current value and the value from the last reading.
- A pump status card showing the actuator's current state (ON/OFF) and a manual override control.
- A 2x2 grid of line charts, each with a 10/30/60 point range selector.

History page layout:

- Date range picker and sensor filter dropdown.
- Four average statistic cards.
- Four historical line charts, one per sensor, plus a log of actuator activations over the selected period.

The Telegram bot has no dashboard-style layout — it is a lightweight push channel. It sends a message automatically when an alert condition is detected (FR8), and optionally accepts a small set of commands (e.g. a status request or a manual pump toggle) that map onto the same Service Layer endpoints used by the dashboard, so both interfaces stay consistent with a single source of truth.

4.8 Flow of Operation

To make the end-to-end behavior of the system easy to follow, the full data and decision flow is summarized below:

Sensors + Actuator → ESP8266 (Virtualization Layer) → MQTT Broker → Subscriber + Alert Engine (Digital Twin Layer) → Database → Flask API (Service Layer) → Dashboard / Telegram Bot (Application Layer)

Step by step: the ESP8266 reads the four sensors on a fixed cycle and publishes each reading to its own MQTT topic. The Mosquitto broker distributes each message to every subscriber independently, without the ESP8266 needing to know who is listening. The Python subscriber persists every reading to the database and, in the same step, evaluates it against the configured thresholds. If a critical condition is found, three things can happen in parallel: the alert engine marks the corresponding sensor as "warn" or "danger" for the dashboard (FR6), it publishes an actuator command if the condition calls for a physical response (FR7), and it dispatches a Telegram message to the user (FR8). Meanwhile, the Flask API continuously serves the latest and historical data to the dashboard on request, independent of whether an alert is currently active, which is what allows the dashboard to fall back to its last known state if the ESP8266 temporarily disconnects (NFR2).

5. Demo Description

This section describes the physical demonstration used to validate the system, rather than the implementation itself, since the source code is reviewed separately.

5.1 Configuration

The demo setup consists of:

- An ESP8266 (NodeMCU v3) connected to the DHT11, the soil moisture sensor, and the LDR light sensor, as described in section 4.4.
- A relay-driven water pump acting as the actuator, controlled automatically by the soil moisture threshold and manually from the dashboard or Telegram bot.
- A laptop running the Mosquitto broker, the Python subscriber, and the Flask server, on the same local WiFi network as the ESP8266.
- The web dashboard, accessed from a browser on the same network.
- A Telegram bot, configured with a chat ID belonging to the demo operator, used to demonstrate FR8.

5.2 Demo Flow

During the demo, sensor values are shown updating live on the dashboard within a few seconds of being published. A critical condition is simulated by manually lowering the soil moisture reading below the configured threshold: this is shown to simultaneously raise a red alert badge on the dashboard, activate the pump (visible through the relay's indicator LED), and deliver a Telegram message to the operator's phone, demonstrating that the same detected event drives all three response channels (FR6, FR7, FR8) from a single source of truth. Returning the reading to a safe value is shown to clear the alert and switch the pump off automatically. Finally, a brief network interruption is simulated by disconnecting the ESP8266, showing that the dashboard falls back gracefully and displays the last known values rather than breaking (NFR2).

6. Conclusion

This project demonstrates how a complete IoT monitoring and response system can be built using low-cost hardware and open-source software. The ESP8266 proved to be a suitable choice for this kind of application — it is affordable, easy to program, and handles WiFi communication natively.

The Publish/Subscribe pattern, implemented through MQTT, worked well for the sensor data transmission and made it straightforward to add the alert engine and actuator control as independent subscribers without touching the code running on the ESP8266. This flexibility is what allowed the system to grow from a passive monitoring tool into one that also acts on the environment and reaches the user proactively.

The web dashboard achieved the goal of providing a clear and readable view of farm conditions in real time, while the addition of the Telegram bot addressed a real gap: a user does not need to keep the dashboard open to be warned of a critical condition. Combined with the actuator, the system moves from reporting problems to helping resolve them automatically.

Working in two people with a hardware/software split was effective but required careful coordination, especially around the message format and the database schema shared between both sides. The decision to write a seed script early on allowed the software side to progress independently without waiting for the physical hardware.